

Peer-to-Peer Campus Network Using Data Structures and Algorithms (Python)

Introduction

This project simulates a mini campus social network where students are connected with each other. The system uses multiple Data Structures and Algorithms concepts such as Hash Tables, Graphs, Binary Search Trees (BST), Recursion, and Sorting algorithms to manage student data efficiently.

Objectives

Objectives:

- Store and retrieve student profiles efficiently.
- Represent friendships between students.
- Find the shortest connection between students.
- Search students efficiently using Binary Search Tree.
- Generate a leaderboard of students based on GPA.

Data Structures Used

Data Structures Used: 1. Hash Table – Used to store student profiles with Student ID as the key. 2. Graph (Adjacency List) – Represents friendships between students. 3. Binary Search Tree – Used to search students efficiently. 4. Recursion – Used for tree traversal operations. 5. Sorting Algorithm (Merge Sort) – Used to rank students by GPA.

Implementation Steps

Implementation Steps: Step 1: Create a Student class to store student ID, name, email and GPA. Step 2: Implement a Hash Table using Python dictionary for quick lookup. Step 3: Build a Graph using adjacency list to represent friendships. Step 4: Use Breadth First Search (BFS) to find shortest connection between students. Step 5: Insert students into a Binary Search Tree based on student ID. Step 6: Traverse the BST using recursion (In-order traversal). Step 7: Use Merge Sort to generate a GPA leaderboard.

Sample Code

Sample Python Code Structure:

```
class Student:
def __init__(self, student_id, name, email, gpa):
self.student_id = student_id
self.name = name
self.email = email
self.gpa = gpa

registry = {}
registry[101] = Student(101, "Alice", "alice@mail.com", 3.8)
```

Complexity Analysis

Component	Operation	Time Complexity
Hash Table	Search Student	$O(1)$
Graph (BFS)	Shortest Path	$O(V + E)$
Binary Search Tree	Search	$O(\log n)$
Merge Sort	Ranking Students	$O(n \log n)$

Conclusion

Conclusion: This project demonstrates how multiple Data Structures and Algorithms can work together to build an efficient campus network system. Hash Tables allow quick data retrieval, Graphs represent relationships between students, BST improves search efficiency, and sorting algorithms help generate student rankings.